



General Description

This document describes how to build and develop Metamask Gardio Snap

Contents

General Description	1
1 Prerequisites	3
1.1.1 Install Node and npm	3
1.1.2 Install yarn.....	3
1.1.3 Versions.....	3
2 Gardio Snap.....	4
2.1 Clone the repository	4
2.1.1 Github Repo	4
2.1.2 npm package	4
2.2 Build	4
2.2.1 Yarn install (Development build).....	4
2.3 npm publish (Production build)	5
2.3.1 Update the Snap Version	5
2.3.2 Build the Snap	5
2.3.3 Login to npm	5
2.3.4 Publish to npm	6
3 Gardio Dapp	7
3.1 Clone the repository	7
3.1.1 Github Repo	7
• https://github.com/ThirdWayv/gardioMetaMaskSnap	7
3.2 Build	7
3.2.1 Yarn install (Development build).....	7
3.3 Dapp Deploy (Production build).....	8
3.3.1 Build the dApp.....	8
3.3.2 Ensure Firebase Tools are installed.....	8
3.3.3 Login to Firebase	8



MetaMask Gardio Snap Development Guide

3.3.4	Deploy to Firebase	8
4	Build Wasm	9
4.1	Prerequisites	9
4.1.1	WSL.....	9
4.1.2	Emscripten	9
4.2	Build	9
5	Solana Support.....	12
5.1	Support in WASM.....	12
5.2	Set Solana Explorer	12
5.3	Solana Transaction.....	12
6	References	14
6.1	MetaMask	14
6.1.1	Snap Document.....	14
•	https://docs.metamask.io/snaps/	14
6.1.2	Snap Simple MetaMask Example Repo.....	14
6.2	Jira Tickets.....	14
6.3	Wasm	14
6.4	Solana Support.....	14

1 Prerequisites

1.1.1 Install Node and npm

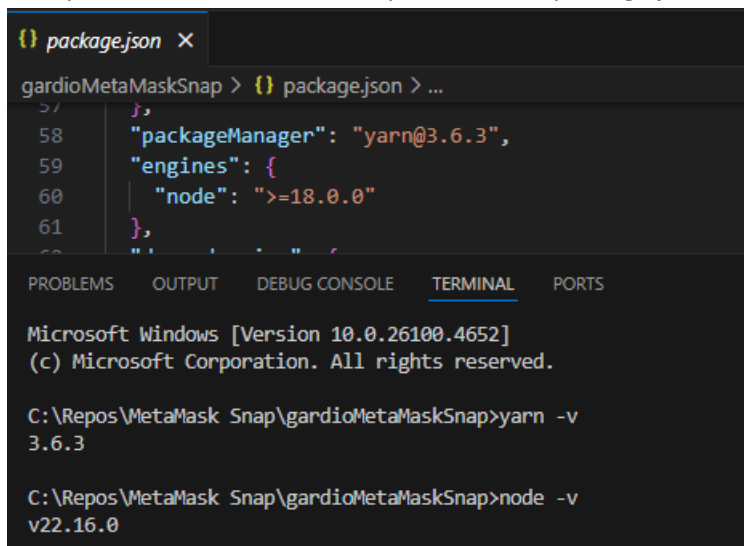
- Download and install Node.js
 - <https://nodejs.org/en>
 - This will also install **npm** (Node Package Manager).
- To verify the installation, run these commands
 - `node -v`
 - `npm -v`

1.1.2 Install yarn

- Node & npm should be installed first
- Run this install command in the terminal
 - `npm install -g yarn`
- To verify the installation, run these commands
 - `yarn -v`

1.1.3 Versions

- We build the Snap and the Dapp with
 - yarn version **3.6.3**
 - node version **22.16.0** ($\geq 18.0.0$)
- Run this command to set our package.json version
 - `yarn set version 3.6.3`
- Verify the versions of node and yarn as in the package.json of your project



The screenshot shows a VS Code editor with a file named `package.json` open. The file content is as follows:

```
57 {  
58   "packageManager": "yarn@3.6.3",  
59   "engines": {  
60     "node": ">=18.0.0"  
61   },  
62 }
```

Below the editor, the TERMINAL tab is active, showing the following output:

```
Microsoft Windows [Version 10.0.26100.4652]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Repos\MetaMask Snap\gardioMetaMaskSnap>yarn -v  
3.6.3  
  
C:\Repos\MetaMask Snap\gardioMetaMaskSnap>node -v  
v22.16.0
```

2 Gardio Snap

2.1 Clone the repository

2.1.1 Github Repo

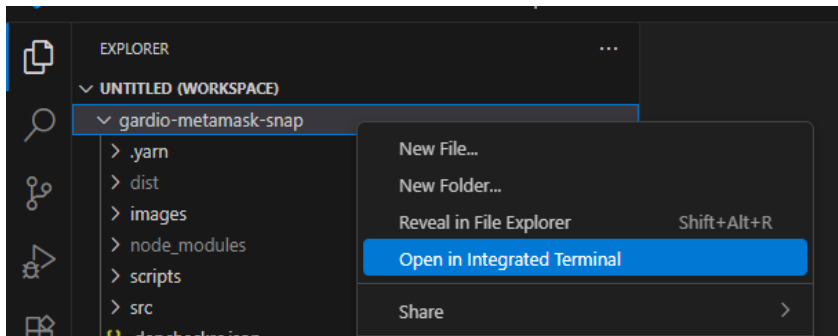
- <https://github.com/ThirdWayv/gardio-metamask-snap>

2.1.2 npm package

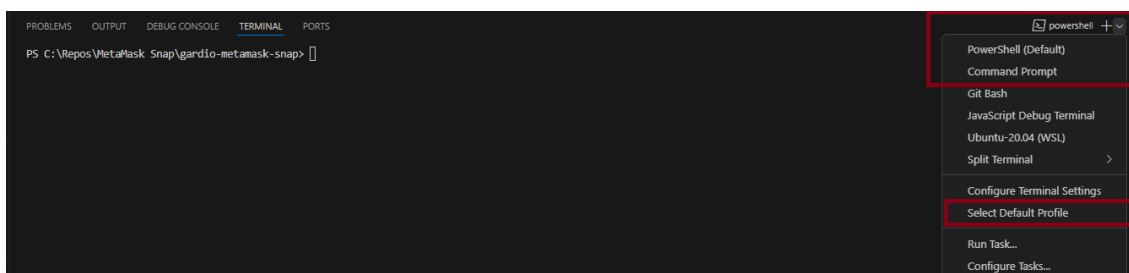
- <https://www.npmjs.com/package/@gardio/gardio-snap>

2.2 Build

- Clone the GitHub repo
- Add the folder to VS Code workspace
- Open Visual Studio Code terminal for this folder



- As default terminal in VS Code will open as PowerShell
 - You will need to switch to Command Prompt (CMD)
 - You can make it the Default terminal for VS Code



2.2.1 Yarn install (Development build)

- Run this command first
 - yarn
 - the expected output:

```
C:\Repos\MetaMask Snap\gardio-metamask-snap>yarn
YN0000: Resolution step
> YN0002: @gardio/gardio-snap@workspace:. doesn't p
> YN0002: @gardio/gardio-snap@workspace:. doesn't p
> YN0002: @gardio/gardio-snap@workspace:. doesn't p
> YN0060: @gardio/gardio-snap@workspace:. provides
> YN0060: @gardio/gardio-snap@workspace:. provides
> YN0060: @gardio/gardio-snap@workspace:. provides
> YN0060: @gardio/gardio-snap@workspace:. provides
```

- Run this command to start developing

- yarn start
- the expected output:

```
YN0000: Completed in 9s 493ms
> YN0000: Done with warnings in 10s 180ms

C:\Repos\MetaMask Snap\gardio-metamask-snap>yarn start
i The server is listening on http://localhost:8080.
i Snap bundle evaluated successfully.
i Compiled 494 files in 9207ms.
✓ Done!
i Changes detected in C:\Repos\MetaMask Snap\gardio-metamask-snap\snap.manifest.json, recompiling.
i Snap bundle evaluated successfully.
i Compiled 494 files in 12005ms.
✓ Done!
```

- When you develop the code, the terminal will detect the change automatically and recompile

2.3 npm publish (Production build)

2.3.1 Update the Snap Version

- npm does not allow publishing the same version twice.
- Before publishing, **update the version** in both:
 - package.json
 - snap.manifest.json
- Ensure the version numbers **match exactly** in both files.

```
{ } package.json x
gardio-metamask-snap > { } package.json > ...
1 {
2   "name": "@gardio/gardio-snap",
3   "version": "1.1.0",
```

```
{ } snap.manifest.json M x
gardio-metamask-snap > { } snap.manifest.json > ...
1 {
2   "version": "1.1.0",
```

2.3.2 Build the Snap

- Run this command to build snap
 - yarn build

2.3.3 Login to npm

- You only need to log in once per environment or if you've been logged out.
- We use the following credentials for publishing:
 - Email: software.team@thirdwayv.com



MetaMask Gardio Snap Development Guide

- Username: `thirdwayv_developer`
- To log in, run:
 - `npm login`

2.3.4 Publish to npm

- Run this command to npm publish
 - `npm publish`

3 Gardio Dapp

3.1 Clone the repository

3.1.1 Github Repo

- <https://github.com/ThirdWayv/gardioMetaMaskSnap>

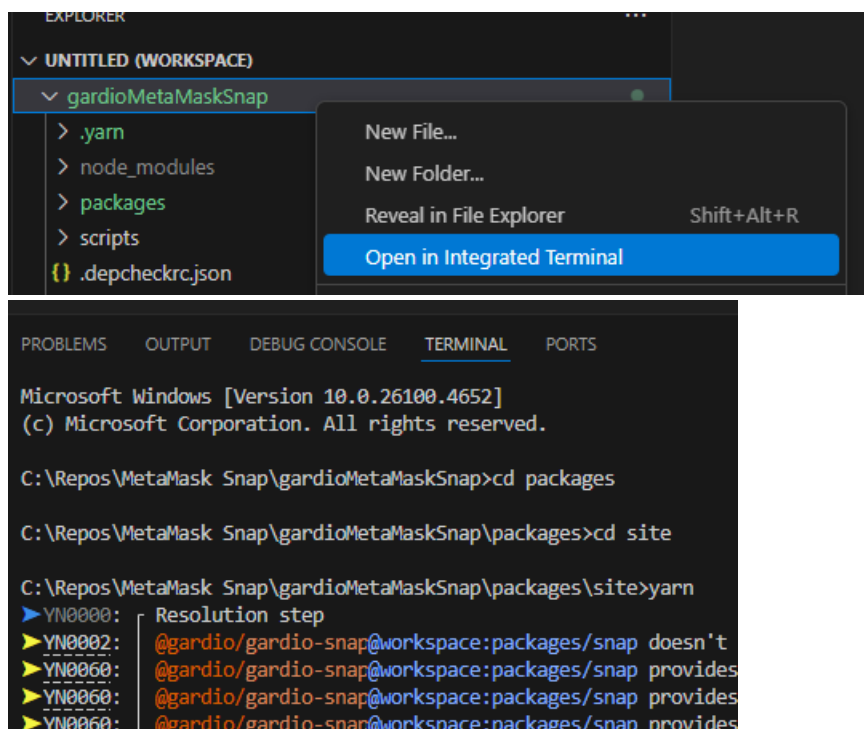
3.2 Build

- Clone the GitHub repo
- Add the folder to VS Code workspace
- Open Visual Studio Code terminal for this folder

3.2.1 Yarn install (Development build)

- Run this command in this path in the repo “gardioMetaMaskSnap/packages/site”

○ yarn



```
EXPLORER
└─ UNTITLED (WORKSPACE)
  └─ gardioMetaMaskSnap
    ├── .yarn
    ├── node_modules
    ├── packages
    ├── scripts
    └── .depcheckrc.json

New File...
New Folder...
Reveal in File Explorer (Shift+Alt+R)
Open in Integrated Terminal

TERMINAL
Microsoft Windows [Version 10.0.26100.4652]
(c) Microsoft Corporation. All rights reserved.

C:\Repos\MetaMask Snap\gardioMetaMaskSnap>cd packages
C:\Repos\MetaMask Snap\gardioMetaMaskSnap\packages>cd site
C:\Repos\MetaMask Snap\gardioMetaMaskSnap\packages\site>yarn
> YN0000: Resolution step
> YN0002: @gardio/gardio-snap@workspace:packages/snap doesn't
> YN0060: @gardio/gardio-snap@workspace:packages/snap provides
> YN0060: @gardio/gardio-snap@workspace:packages/snap provides
> YN0060: @gardio/gardio-snap@workspace:packages/snap provides
```

- Run this command and open the local Dapp link

○ yarn start

○ <http://localhost:8000/>

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

You can now view @metamask/snap-simple-keyring-site in the browser.

http://localhost:8000/

View GraphQL, an in-browser IDE, to explore your site's data and schema

http://localhost:8000/___graphql

Note that the development build is not optimized.
To create a production build, use gatsby build

success Building development bundle - 37.814s
success Writing page-data.json and slice-data.json files to public directory - 0.337s - 2/8 23.76/s
```

- Then go to “**gardio-MetaMask-Snap-Guide**” Document to know how to use it

3.3 Dapp Deploy (Production build)

3.3.1 Build the dApp

- yarn build

3.3.2 Ensure Firebase Tools are installed

- yarn add -g firebase-tools

3.3.3 Login to Firebase

- You only need to do this once per environment or if you're logged out.
- We are using omar.sabry@thirdwayv.com for deploying to Firebase.
 - Firebase login

3.3.4 Deploy to Firebase

- firebase deploy

4 Build Wasm

4.1 Prerequisites

4.1.1 WSL

- Must have WSL first
- If you don't have it, follow this document
 - <https://thirdwayv.atlassian.net/wiki/spaces/CRYP/pages/2840461313/Automation+test+bri+ngup>

4.1.2 Emscripten

- Clone the Emscripten SDK
 - <https://github.com/emscripten-core/emsdk>
- Open WSL terminal in emsdk path and run

- `./emsdk install 3.1.64`
- `./emsdk activate 3.1.64`

```
samuel_george@TWI-1K19RG3:/mnt/c/Repos/emscdk$ ./emsdk install 3.1.64
Resolving SDK version '3.1.64' to 'sdk-releases-fd61bacaf40131f74987e649a135f1dd559aff60-64bit'
Installing SDK 'sdk-releases-fd61bacaf40131f74987e649a135f1dd559aff60-64bit'..
Skipped installing node-22.16.0-64bit, already installed.
Skipped installing releases-fd61bacaf40131f74987e649a135f1dd559aff60-64bit, already installed.
All SDK components already installed: 'sdk-releases-fd61bacaf40131f74987e649a135f1dd559aff60-64bit'.
samuel_george@TWI-1K19RG3:/mnt/c/Repos/emscdk$ ./emsdk activate 3.1.64
Resolving SDK version '3.1.64' to 'sdk-releases-fd61bacaf40131f74987e649a135f1dd559aff60-64bit'
Setting the following tools as active:
  node-22.16.0-64bit
  releases-fd61bacaf40131f74987e649a135f1dd559aff60-64bit

Next steps:
- To conveniently access emsdk tools from the command line,
  consider adding the following directories to your PATH:
  /mnt/c/Repos/emscdk
  /mnt/c/Repos/emscdk/node/22.16.0_64bit/bin
  /mnt/c/Repos/emscdk/upstream/emscripten
- This can be done for the current shell by running:
  source "/mnt/c/Repos/emscdk/emscdk_env.sh"
- Configure emsdk in your shell startup scripts by running:
  echo 'source "/mnt/c/Repos/emscdk/emscdk_env.sh"' >> $HOME/.bash_profile
samuel_george@TWI-1K19RG3:/mnt/c/Repos/emscdk$
```

4.2 Build

- Go to emsdk path and open WSL terminal
- Run this command
 - `source emsdk_env.sh`

MetaMask Gardio Snap Development Guide

```
samuel_george@TWI-1K19RG3:/mnt/c/Repos/emsdk$ source emsdk_env.sh
Setting up EMSDK environment (suppress these messages with EMSDK_QUIET=1)
Adding directories to PATH:
PATH += /mnt/c/Repos/emsdk
PATH += /mnt/c/Repos/emsdk/upstream/emscripten
PATH += /mnt/c/Repos/emsdk/node/22.16.0_64bit/bin

Setting environment variables:
PATH = /mnt/c/Repos/emsdk:/mnt/c/Repos/emsdk/upstream/emscripten:/mnt/c/Repos/emsdk/node/22.16.0_64bit/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux.2.5.7.0_x64__8wekyb3d8bbwe:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/TortoiseGit/bin:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/Microsoft VS Code/bin:/mnt/c/Program Files/usbipd-win:/mnt/c/Program Files/nodejs:/mnt/c/Program Files/CMake/bin:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/Nordic Semiconductor/nrf-command-line-tools/bin:/mnt/c/Users/SultanArabi/AppData/Local/Programs/Python/Python311:/mnt/c/Users/SultanArabi/AppData/Local/Programs/Python/Python311/Scripts:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/Users/SamuelGeorge/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/SamuelGeorge/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/SamuelGeorge/AppData/Roaming/npm:/mnt/c/Ruby30-x64/bin:/mnt/c/Users/SamuelGeorge/AppData/Local/Programs/Python/Python313:/mnt/c/Users/SamuelGeorge/AppData/Local/Programs/Python/Python313/Scripts:/mnt/c/msys64/ucrt64/bin:/mnt/c/Users/SamuelGeorge/.dotnet/tools:/snap/bin
EMSDK = /mnt/c/Repos/emsdk
EMSDK_NODE = /mnt/c/Repos/emsdk/node/22.16.0_64bit/bin/node
samuel_george@TWI-1K19RG3:/mnt/c/Repos/emsdk$
```

- Go to the MetaMask snap CMake Path in the WalletCore “**WalletCore/Build/MetaMask Snap**”
 - You will find the correct path and CMake file that builds in **CRYP-4449 Branch only**
 - <https://github.com/ThirdWayv/TWIWalletCore/tree/CRYP-4449>
- Ensure you’re in the same terminal where you’ve run this command “`source emsdk_env.sh`”

```
samuel_george@TWI-1K19RG3:/mnt/c/Repos/emsdk$ cd ../MetaMask\ Snap/CRYP-4449/Build/MetaMask\ Snap/
samuel_george@TWI-1K19RG3:/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap$
```

- Run cmake command
 - emcmake cmake

```
samuel_george@TWI-1K19RG3:/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap$ emcmake cmake
configure: cmake -DCMAKE_TOOLCHAIN_FILE=/mnt/c/Repos/emsdk/upstream/emscripten/cmake/Modules/Platform/Emscripten.cmake -DCMAKE_CROSSCOMPILING_EMULATOR=/mnt/c/Repos/emsdk/node/22.16.0_64bit/bin/node
CMake Warning:
  No source or binary directory provided. Both will be assumed to be the
  same as the current working directory, but note that this warning will
  become a fatal error in future CMake releases.

-- GIT_COMMIT="daa4041d"
-- Configuring done
-- Generating done
-- Build files have been written to: /mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap
samuel_george@TWI-1K19RG3:/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap$
```

- If you run into this error, you need to clean up first

```
samuel_george@TWI-1K19RG3:/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap$ emcmake cmake
configure: cmake -DCMAKE_TOOLCHAIN_FILE=/mnt/c/Repos/emsdk/upstream/emscripten/cmake/Modules/Platform/Emscripten.cmake -DCMAKE_CROSSCOMPILING_EMULATOR=/mnt/c/Repos/emsdk/node/22.16.0_64bit/bin/node
CMake Warning:
  No source or binary directory provided. Both will be assumed to be the
  same as the current working directory, but note that this warning will
  become a fatal error in future CMake releases.

CMake Error: The current CMakeCache.txt directory /mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap/CMakeCache.txt is different than the directory /mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Extension where CMakeCache.txt was created. This may result in binaries being created in the wrong place. If you are not sure, reedit the CMakeCache.txt
CMake Error: The source "/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap/CMakeLists.txt" does not match the source "/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Extension/CMakeLists.txt" used to generate cache. Re-run cmake with a different source directory.
emcmake: error: 'cmake -DCMAKE_TOOLCHAIN_FILE=/mnt/c/Repos/emsdk/upstream/emscripten/cmake/Modules/Platform/Emscripten.cmake -DCMAKE_CROSSCOMPILING_EMULATOR=/mnt/c/Repos/emsdk/node/22.16.0_64bit/bin/node' failed (returned 1)
samuel_george@TWI-1K19RG3:/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap$
```

- Run make command
 - make -j20

```
1 warning generated.
[100%] Linking C executable crypto_guard_usb_if.js
warning: undefined symbol: consoleLog (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onConnectionDone (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onGenuineCheckResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onGetAddressResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onGetInstalledAppsInfoResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onGetWalletIdResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onGetXpubResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onInstallAppResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onOperationCleanUp (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onSessionChallenge (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onSessionChallengeRsp (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onSignMsgResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onSignTxResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onSignTypedDataResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onTerminate (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onUninstallAppResult (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: onUserConfirmation (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: requestUserConfirmation (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: startTimer (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: stopTimer (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: usbConnect (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: usbReadFeatureReport (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: usbSendFeatureReport (referenced by root reference (e.g. compiled C/C++ code))
warning: undefined symbol: usbSendReport (referenced by root reference (e.g. compiled C/C++ code))
emcc: warning: warnings in JS library compilation [-Wjs-compiler]
[100%] Built target crypto_guard_if
samuel_george@TWI-1K19RG3:/mnt/c/Repos/MetaMask Snap/CRYP-4449/Build/MetaMask Snap$
```

- You will find all callbacks that WASM needs the app to implement
- Then you will find the wasm in the same path **“WalletCore/Build/MetaMask Snap”**
- Copy **“crypto_guard_usb_if.wasm”** to Dapp path
 - **“gardioMetaMaskSnap\packages\site\public”**
- Rebuild the Dapp

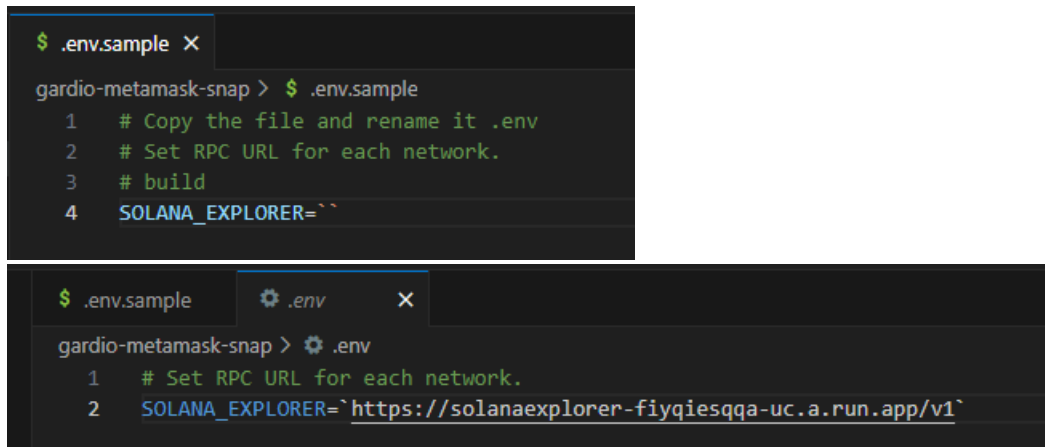
5 Solana Support

5.1 Support in WASM

- You will need to define `SOLANA\_SUPPORTED` flag in CMakeLists.txt
- Add `twi\_solana\_json\_parser` in the paths
- Exist in Branch CRYP-4449 in WalletCore
 - <https://github.com/ThirdWayv/TWIWalletCore/tree/CRYP-4449>

5.2 Set Solana Explorer

- In branches that have .env.sample as CRYP-4449 & CRYP-4505 will need to copy it and rename with .env
- Then set the Solana explorer RPC URL
- If the branch doesn't have .env.sample, so no Solana and no need for .env
- Then build the snap

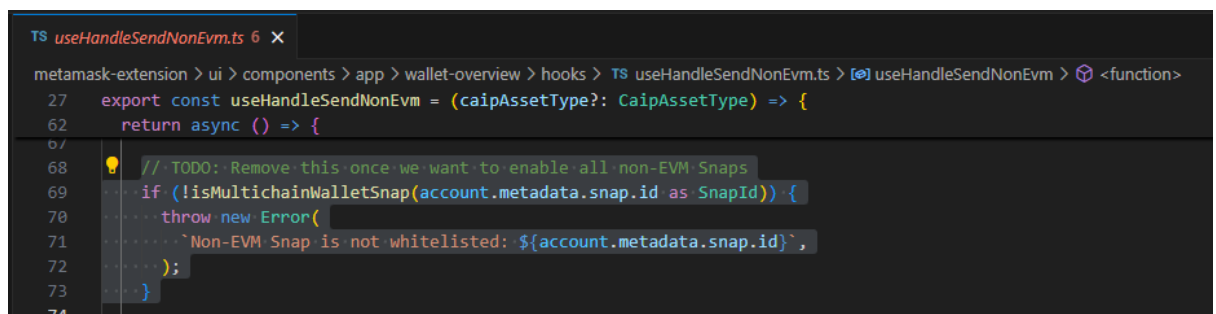


```
$ .env.sample X
gordio-metamask-snap > $ .env.sample
1 # Copy the file and rename it .env
2 # Set RPC URL for each network.
3 # build
4 SOLANA_EXPLORER=`

$ .env.sample .env X
gordio-metamask-snap > .env
1 # Set RPC URL for each network.
2 SOLANA_EXPLORER=`https://solanaexplorer-fiyqiesqqa-uc.a.run.app/v1`
```

5.3 Solana Transaction

- Solana Transactions is not supported yet in MetaMask for the Non-EVM snaps, as the gardio snap
- Need to build extension and comment the block in the Metamask extension
- <https://github.com/MetaMask/metamask-extension>
- Open tags and the latest metamask flask extension version is 12.22.2, and not support solana tx yet
- Download source code to build
 - Go to Metamask Extension Development Guide
 - Remove this block



```
TS useHandleSendNonEvm.ts 6 X
metamask-extension > ui > components > app > wallet-overview > hooks > TS useHandleSendNonEvm.ts > useHandleSendNonEvm > <function>
27 export const useHandleSendNonEvm = (caipAssetType?: CaipAssetType) => {
62   return async () => {
63     //
64     // TODO: Remove this once we want to enable all non-EVM Snaps
65     if (!isMultichainWalletSnap(account.metadata.snap.id as SnapId)) {
66       throw new Error(
67         `Non-EVM Snap is not whitelisted: ${account.metadata.snap.id}`,
68       );
69     }
70   }
71 }
72
73
74
```

- Or can just download the output extension and remove the block in the output extension
 - Download **Metamask-flask-chrome-12.22.2**
 - Remove this line

```
metamask-flask-chrome-12.22.2-flask.0 (1) > JS ui-5.js > <function> > <function> > <function> > useHandleSendNonEvm > <function>
routes");n.useHandleSendNonEvm=e=>{const{nativeCurrency:t}=(0,o.useSelector)(c.getSelectedMultiChainNetworkConfiguration),n=(0,o.useSelector)(1.getSelected
f=(0,o.useSelector)((e=>e.metamask.defaultHomeActiveTabName)),h=(0,o.useSelector)(1.getMemoizedUnapprovedTemplatedConfirmations);return(0,r.useEffect)((()
snap&&n.metadata.snap.id==e.origin&&(0,u.isMultiChainWalletSnap)(n.metadata.snap.id));e&&p.push(`${d.CONFIRMATION_V_NEXT_ROUTE}/${e.id}`)}},[h,p,n]],asy
be Snap accounts");if(!((0,u.isMultiChainWalletSnap)(n.metadata.snap.id))throw new Error("Non-EVM Snap is not whitelisted: ${n.metadata.snap.id}");const r=
provided, and could not find a fallback native asset for the selected account");return t}}(),{chainId:o}=(0,a.parseCaipAssetType)(r);try{await m((0,s.setT
sendMultiChainTransaction)(n.metadata.snap.id,{account:n.id,scope:o,assetType:r})}catch(e){await m((0,s.setDefaultHomeActiveTabName)(f))}})}},{package:"$
```

Releases / v12.22.2

Version 12.22.2

github-actions released this last week v12.22.2 f21f2f3

Fixed

- Fix crash on older browsers (#34255)

Assets 6

metamask-chrome-12.22.2.zip	sha256:231c1aabfe6171a4d2de4d0a54897...	24.6 MB	last week
metamask-firefox-12.22.2.zip	sha256:1e456498eed5bbf167dc88dc23f22...	24.5 MB	last week
metamask-flask-chrome-12.22.2-flask.0.zip	sha256:713d8dea4ec1884f0024561f96749...	25.9 MB	last week
metamask-flask-firefox-12.22.2-flask.0.zip	sha256:2936229f959558b77a90d1cc958d3...	25.8 MB	last week
Source code (zip)			last week
Source code (tar.gz)			last week

1 person reacted

- Then, Solana transaction flow is not documented yet in Metamask documentation
- But there is an example repo for Solana official by MetaMask that can be a reference
 - <https://github.com/MetaMask/snap-solana-wallet>
- All Jira tickets that you need to review before starting to develop the Solana transaction in the snap
 - <https://thirdwayv.atlassian.net/browse/CRYP-4440>
 - <https://thirdwayv.atlassian.net/browse/CRYP-4449>
 - <https://thirdwayv.atlassian.net/browse/CRYP-4505>
 - <https://thirdwayv.atlassian.net/browse/CRYP-4504>



6 References

6.1 MetaMask

6.1.1 Snap Document

- <https://docs.metamask.io/snaps/>

6.1.2 Snap Simple MetaMask Example Repo

- <https://github.com/MetaMask/snap-simple-keyring>

6.2 Jira Tickets

- <https://thirdwayv.atlassian.net/issues/?jql=project%20%3D%20%27CRYP%27%20AND%20textfields%20~%20%22MetaMask%20Snap%22>

6.3 Wasm

- <https://github.com/emscripten-core/emsdk>
- <https://thirdwayv.atlassian.net/wiki/spaces/CRYP/pages/2840461313/Automation+test+bringup>

6.4 Solana Support

- <https://github.com/MetaMask/snap-solana-wallet>
- <https://www.npmjs.com/package/@metamask/solana-wallet-snap>